iroh-blob 오류 처리 및 복구 전략 분석

개요

iroh-blob은 분산 환경에서 안정적인 데이터 전송을 위해 포괄적인 오류 처리 및 복구 메커니즘을 구현합니다. 이 문서는 네트워크 오류, 데이터 무결성 오류, 저장소 오류의 분류와 처리 방식, 그리고 연결 중단 시 복구 메커니즘을 상세히 분석합니다.

오류 분류 체계

1. 주요 오류 타입 계층 구조

iroh-blob의 오류 처리는 `GetError` 를 중심으로 한 계층적 구조를 가집니다:

```
pub  enum GetError {

/// 해시를 찾을 수 없거나 요청한 청크를 찾을 수 없음

NotFound { source: Box<NotFoundCases> },

/// 원격 노드가 연결을 재설정함

RemoteReset { source: Box<RemoteResetCases> },

/// 원격 노드가 비준수 방식으로 동작함

NoncompliantNode { source: Box<NoncompliantNodeCases> },

/// 네트워크 또는 I/O 작업 실패

Io { source: Box<IoCases> },

/// 잘못된 다운로드 요청

BadRequest { source: Box<BadRequestCases> },
```

```
/// 로컬 노드에서 작업 실패
LocalFailure { source: Box<LocalFailureCases> },
}
```

2. 네트워크 오류 분류

2.1 연결 수준 오류 (ConnectionError)

```
// QUIC 연결 오류의 분류와 처리 전략
match  connection_error {
ConnectionError::VersionMismatch => {
// 지원되지 않는 버전 - 장기적 오류로 분류
// 해당 피어는 사용 불가능으로 표시
NoncompliantNodeSnafu.into_error(e.into())
}
ConnectionError::TransportError(_) => {
// QUIC 사양 위반 - 비준수 노드로 분류
NoncompliantNodeSnafu.into_error(e.into())
}
ConnectionError::ConnectionClosed(_) => {
// 피어의 QUIC 스택이 자동으로 연결 중단
IoSnafu.into_error(e.into())
}
ConnectionError::Reset => {
```

```
// 피어가 재시작으로 인해 연결 처리 불가
RemoteResetSnafu.into_error(e.into())
}
ConnectionError::TimedOut => {
// 유휴 타임아웃 초과
IoSnafu.into_error(e.into())
}
}
```

2.2 스트림 수준 오류

읽기 오류 (ReadError):

- `ReadError::Reset(_)` : 원격 재설정으로 분류
- `ReadError::ConnectionLost` : 연결 손실, 상위 연결 오류로 전파
- `ReadError::ClosedStream` : I/O 오류로 분류, 복구 가능

쓰기 오류 (WriteError):

- `WriteError::Stopped(_)` : 원격 중지, 재설정으로 분류
- `WriteError::ConnectionLost` : 연결 손실, 상위 연결 오류로 전파
- `WriteError::ClosedStream` : I/O 오류로 분류, 복구 가능

3. 데이터 무결성 오류

3.1 DecodeError 분류

```
pub enum DecodeError {
```

```
/// 청크를 찾을 수 없음
ChunkNotFound,
/// 부모 노드를 찾을 수 없음
ParentNotFound,
/// 리프 노드를 찾을 수 없음
LeafNotFound,
/// 부모 해시 불일치
ParentHashMismatch { expected: Hash, actual: Hash },
/// 리프 해시 불일치
LeafHashMismatch { expected: Hash, actual: Hash },
/// 읽기 오류
Read { source: ReadError },
/// 디코딩 I/O 오류
DecodeIo { source: io::Error },
}
```

3.2 해시 검증 실패 처리

해시 불일치 오류:

- `ParentHashMismatch` : 비준수 노드로 분류, 데이터 손상 또는 잘못된 요청 가능성
- `LeafHashMismatch` : 비준수 노드로 분류, 실시간 검증 실패

데이터 누락 오류:

- `ChunkNotFound` : NotFound로 분류, 나중에 재시도 가능
- `ParentNotFound` / `LeafNotFound` : NotFound로 분류, 부분 데이터 상황

4. 저장소 오류

4.1 로컬 실패 케이스 (LocalFailureCases)

```
pub enum LocalFailureCases {
/// 일반 I/O 오류
Io { source: io::Error },
/// 일반적인 anyhow 오류
Anyhow { source: anyhow::Error },
/// IRPC 채널 전송 오류
IrpcSend { source: irpc::channel::SendError },
/// IRPC 오류
Irpc { source: irpc::Error },
/// BAO 내보내기 오류
ExportBao { source: ExportBaoError },
/// Tokio 채널 전송 오류
TokioSend {},
}
```

4.2 파일 시스템 저장소 특화 오류

메타데이터 액터 오류:

```
pub enum ActorError {
/// 테이블 오류
Table { source: redb::TableError },
```

```
/// 트랜잭션 오류
Transaction { source: redb::TransactionError },
/// 데이터베이스 오류
Database { source: redb::DatabaseError },
}
```

복구 전략 및 메커니즘

1. 연결 중단 시 복구 메커니즘

1.1 연결 풀 기반 복구

```
pub struct Options {
pub idle_timeout: Duration, // 기본 5초
pub connect_timeout: Duration, // 기본 1초
pub max_connections: usize, // 기본 1024
}
```

연결 관리 전략:

- 유휴 타이머를 통한 자동 연결 정리
- 연결 재사용을 통한 효율성 향상
- 최대 연결 수 제한으로 리소스 보호

1.2 자동 재연결 로직

```
// 연결 오류 시 자동 분류 및 처리
impl From<endpoint::ConnectionError> for GetError {
fn  from(value: endpoint::ConnectionError) -> Self {
match  value {
// 복구 가능한 오류들은 IoSnafu로 분류
ConnectionError::ConnectionClosed(_) |
ConnectionError::TimedOut |
ConnectionError::LocallyClosed => {
IoSnafu.into_error(e.into())
}
// 재설정 오류는 별도 처리
ConnectionError::Reset => {
RemoteResetSnafu.into_error(e.into())
}
// 비준수 노드는 장기적 제외
ConnectionError::VersionMismatch |
ConnectionError::TransportError(_) => {
NoncompliantNodeSnafu.into_error(e.into())
}
}
}
}
```

2. 부분 전송 실패 시 상태 복구

2.1 청크 기반 상태 관리

비트필드를 통한 진행 상황 추적:

```
pub struct Bitfield {
// 다운로드된 청크들의 비트맵
// 각 비트는 1024바이트 청크의 완료 상태를 나타냄
}
```

부분 다운로드 복구 과정:

1. 기존 부분 데이터의 무결성 검증
2. 누락된 청크 범위 계산
3. 새로운 GetRequest로 누락 부분만 요청
4. 기존 데이터와 새 데이터 병합

2.2 파일 시스템 저장소의 지속성 복구

```
async fn test_import_bao_persistence_recover() -> TestResult<()> {
// 1. 부분 가져오기 시작
// 2. 프로세스 중단 시뮬레이션
// 3. 재시작 후 상태 복구 확인
// 4. 나머지 데이터 계속 가져오기
}
```

복구 메커니즘:

- 메타데이터 데이터베이스를 통한 상태 지속성
- 부분 파일의 무결성 검증
- 중단된 지점부터 재개 가능

3. 재시도 로직 및 백오프 전략

3.1 오류별 재시도 정책

즉시 재시도 가능한 오류:

- `IoError` : 네트워크 일시적 문제
- `NotFound` : 데이터가 나중에 사용 가능할 수 있음

재시도 불가능한 오류:

- `NoncompliantNode` : 피어 제외 필요
- `BadRequest` : 요청 자체의 문제

조건부 재시도:

- `RemoteReset` : 피어 재시작 후 재시도 가능

3.2 백오프 및 회로 차단기 패턴

```
// 연결 풀에서의 타임아웃 처리
.timeout(context.options.connect_timeout)
.await
.map_err(|_| PoolConnectError::Timeout)
```

지수 백오프 구현:

- 초기 지연: 100ms
- 최대 지연: 30초

- 지수 증가율: 2.0
- 지터 추가로 동시 재시도 방지

4. 상태 기반 오류 복구

4.1 유한 상태 머신의 오류 처리

Get 모듈의 상태별 오류 처리:

```
// AtInitial -> AtConnected 전환 시 오류
impl From<ConnectedNextError> for GetError {
fn  from(value: ConnectedNextError) -> Self {
match  value {
PostcardSer { .. } | RequestTooBig { .. } => {
// 직렬화 오류는 요청 문제
BadRequestSnafu.into_error(e.into())
}
Write { source, .. } | Closed { source, .. } => {
// 네트워크 오류는 상위로 전파
source.into()
}
Io { .. } => {
// I/O 오류는 복구 가능
IoSnafu.into_error(e.into())
}
}
```

```
}

}
```

4.2 엔티티 매니저의 복구 메커니즘

```
pub trait Reset: Default {

/// 상태를 기본값으로 재설정

fn reset(&mut self);

}
```

엔티티 액터 복구:

- 패닉 발생 시 전체 엔티티 비활성화
- 복구 불가능한 상태로 표시
- 새로운 태스크 스폰 시 Dead 상태 반환

모니터링 및 메트릭

1. 오류 추적 메트릭

```
pub struct Metrics {

/// 다운로더 액터의 전송 실패 틱 수

pub downloader_tick_transfer_failed: Counter,

/// 다운로더 액터의 재시도 노드 틱 수

pub downloader_tick_retry_node: Counter,
```

```
/// 다운로더 액터의 작별 노드 틱 수
pub  downloader_tick_goodbye_node: Counter,
}
```

2. 진단 및 디버깅

오류 컨텍스트 보존:

- `SpanTrace` : 비동기 스택 추적
- `Backtrace` : 동기 스택 추적
- 구조화된 오류 정보

로깅 전략:

- 오류 수준별 로그 레벨 차등 적용
- 복구 가능한 오류: WARN 레벨
- 복구 불가능한 오류: ERROR 레벨
- 디버그 정보: DEBUG/TRACE 레벨

보안 고려사항

1. 서비스 거부 공격 방어

리소스 제한:

- 최대 메시지 크기: 100MiB
- 연결당 최대 스트림 수 제한
- 메모리 사용량 모니터링

오류 정보 노출 제한:

- 내부 구현 세부사항 숨김
- 일반화된 오류 메시지 제공
- 민감한 경로 정보 마스킹

2. 데이터 무결성 보장

실시간 검증:

- BLAKE3 해시를 통한 청크별 검증
- 최대 16KiB 지연으로 무결성 확인
- 손상된 데이터 즉시 거부

성능 영향 분석

1. 오류 처리 오버헤드

메모리 사용량:

- 오류 구조체의 박싱을 통한 메모리 최적화
- 스택 추적 정보의 선택적 수집
- 오류 컨텍스트의 지연 평가

CPU 오버헤드:

- 오류 분류 로직의 최적화
- 빠른 경로와 느린 경로 분리
- 인라인 최적화 적용

2. 복구 성능

연결 재사용:

- 기존 연결 풀 활용
- 새 연결 설정 비용 최소화
- 지능적 로드 밸런싱

부분 복구 효율성:

- 청크 단위 세밀한 복구
- 불필요한 데이터 재전송 방지
- 병렬 복구 작업 지원

결론

iroh-blob의 오류 처리 및 복구 전략은 다음과 같은 특징을 가집니다:

1. **계층적 오류 분류**: 복구 가능성에 따른 체계적 분류
2. **상태 기반 복구**: FSM을 통한 정확한 상태 관리
3. **부분 복구 지원**: 청크 단위의 세밀한 복구 메커니즘
4. **성능 최적화**: 오류 처리 오버헤드 최소화
5. **보안 고려**: DoS 공격 방어 및 정보 노출 방지

이러한 포괄적인 오류 처리 시스템을 통해 iroh-blob은 불안정한 네트워크 환경에서도 안정적이고 효율적인 데이터 전송을 보장합니다.